



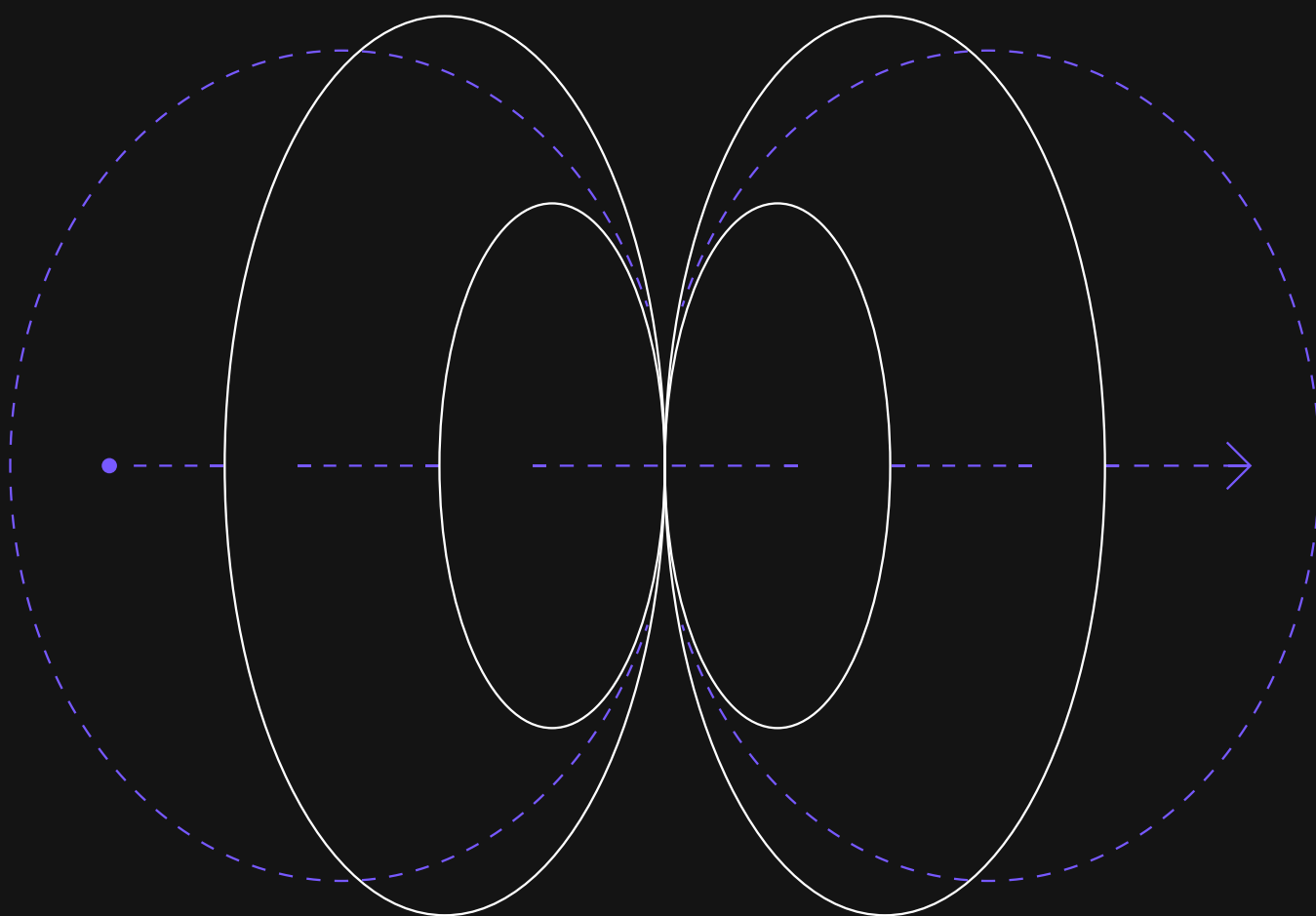
Машинное обучение

Лонгрид

Глубокое обучение

02

Стабильность и регуляризация



Содержание

1. Почему градиенты затухают или взрываются	2
1.1. Обратный проход через полносвязный слой	3
1.2. Откуда берётся нестабильность	4
1.3. Сигмоида как пример затухания	4
2. Функции активации	5
2.1. Возможность сохранить тождество	6
2.2. Вычислительная сложность	6
2.3. Гладкость	6
3. Нормализация	6
3.1. Зачем нормировать промежуточные признаки	6
3.2. Batch Normalization	6
3.3. Обучение и тестирование	7
3.4. Layer Normalization	8
3.5. Где размещать нормализацию	8
4. Инициализация весов	8
4.1. Инициализация Xavier/Glorot	9
4.2. Почему тот же расчёт возникает для $\tanh$	10
4.3. Ортогональная инициализация	11
4.4. Ограничения через репараметризацию	11
4.4.1. Ортогональная матрица	12
4.4.2. Ограниченный выход	12
4.4.3. Положительный параметр	12
5. Остаточные и плотные связи	12
5.1. Два способа переиспользовать признаки	12
5.2. Остаточная связь и градиент	13
5.3. Что дают прокинутые связи	14
6. Dropout	14
6.1. Коадаптация нейронов	15
6.2. Обучение и тестирование	15
7. Стохастическая и неявная регуляризация	15
7.1. Шум стохастического градиента	15
7.2. Широкие минимумы	16
8. Главное	17

Лекция 3. Стабильность и регуляризация в глубоком обучении

О чём эта лекция

В глубокой сети сигнал и градиент проходят через длинную последовательность преобразований. Из-за произведений матриц весов и производных функций активации их масштаб может быстро уменьшаться или расти. В этой лекции собраны способы сделать обучение устойчивее: выбор активации, нормализация, инициализация, репараметризация и остаточные связи. Затем рассматриваются Dropout и неявная регуляризация, возникающая при стохастической оптимизации.

1. Почему градиенты затухают или взрываются



Обозначения

Используем те же обозначения, что и в предыдущих лекциях:

- i — номер объекта, j — номер признака или координаты;
- ℓ — номер слоя, L — глубина сети, n_ℓ — ширина слоя ℓ ;
- $a^{(0)} = x$ — вход сети, $W^{(\ell)}$ и $b^{(\ell)}$ — веса и смещение слоя ℓ ;
- $z^{(\ell)}$ — преактивация, $a^{(\ell)}$ — активация:

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = g^{(\ell)}(z^{(\ell)});$$

- в классификации $z^{(L)}$ — логиты последнего слоя, после которых вычисляются softmax и функция потерь $\mathcal{L}$;
- $\delta^{(\ell)} := \nabla_{z^{(\ell)}} \mathcal{L}$ — градиент по преактивации слоя ℓ ;
- batch из B объектов записываем **по столбцам**:

$$A^{(\ell)} = \begin{bmatrix} a_1^{(\ell)} & \dots & a_B^{(\ell)} \end{bmatrix} \in \mathbb{R}^{n_\ell \times B}.$$

Функция активации здесь обозначается $g^{(\ell)}$; в лекциях 1 и 2 для неё использовалось обозначение φ_ℓ . Это одна и та же покомпонентная нелинейность.

**Не путать обозначения**

$\delta^{(\ell)}$ используется только для градиента обратного прохода. Сдвиг в Batch Normalization обозначается через β .

1.1. ОБРАТНЫЙ ПРОХОД ЧЕРЕЗ ПОЛНОСВЯЗНЫЙ СЛОЙ

Для скрытого слоя

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = g^{(\ell)}(z^{(\ell)})$$

имеем

$$\delta^{(\ell)} = J_{g^{(\ell)}}(z^{(\ell)})^T \nabla_{a^{(\ell)}} \mathcal{L}.$$

Если функция активации применяется покомпонентно, её якобиан диагонален:

$$\delta^{(\ell)} = \nabla_{a^{(\ell)}} \mathcal{L} \odot g^{(\ell)'}(z^{(\ell)}).$$

Градиенты параметров слоя равны

$$\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} a^{(\ell-1)T}, \quad \nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}.$$

Для соседних слоёв обратный проход записывается рекурсивно:

$$\delta^{(\ell)} = J_{g^{(\ell)}}(z^{(\ell)})^T W^{(\ell+1)T} \delta^{(\ell+1)}.$$

Якобианы активаций здесь **чередуются** с матрицами весов. Переставлять множители или собирать их в два независимых произведения нельзя.



Каждый множитель может как уменьшать, так и увеличивать норму

Рисунок 1. Обратный проход — это чередование транспонированных матриц весов и якобианов активаций; масштаб градиента определяется произведением всех этих множителей

1.2. ОТКУДА БЕРЁТСЯ НЕСТАБИЛЬНОСТЬ

Для согласованных операторных норм получаем оценку

$$\|\nabla_{W^{(\ell)}} \mathcal{L}\| \leq \|a^{(\ell-1)}\| \|\delta^{(L)}\| \left(\prod_{k=\ell+1}^L \|W^{(k)}\| \right) \left(\prod_{k=\ell}^{L-1} \|J_{g^{(k)}}(z^{(k)})\| \right).$$

Из-за длинного произведения возможны два противоположных эффекта:

- если большинство множителей меньше единицы, градиент **затухает**;
- если большинство множителей больше единицы, градиент **взрывается**.

В результате первые слои могут обучаться гораздо медленнее последних либо получать слишком большие обновления.

1.3. СИГМОИДА КАК ПРИМЕР ЗАТУХАНИЯ

Для сигмoиды

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Так как $0 < \sigma(z) < 1$, то

$$0 < \sigma'(z) \leq \frac{1}{4}.$$

Следовательно, для диагонального якобиана сигмoиды

$$\|J_{\sigma}(z^{(k)})\| \leq \frac{1}{4}, \quad \prod_{k=\ell}^{L-1} \|J_{\sigma}(z^{(k)})\| \leq \left(\frac{1}{4}\right)^{L-\ell}.$$

Если нормы матриц весов не компенсируют это убывание, градиенты первых слоёв становятся значительно меньше градиентов последних.

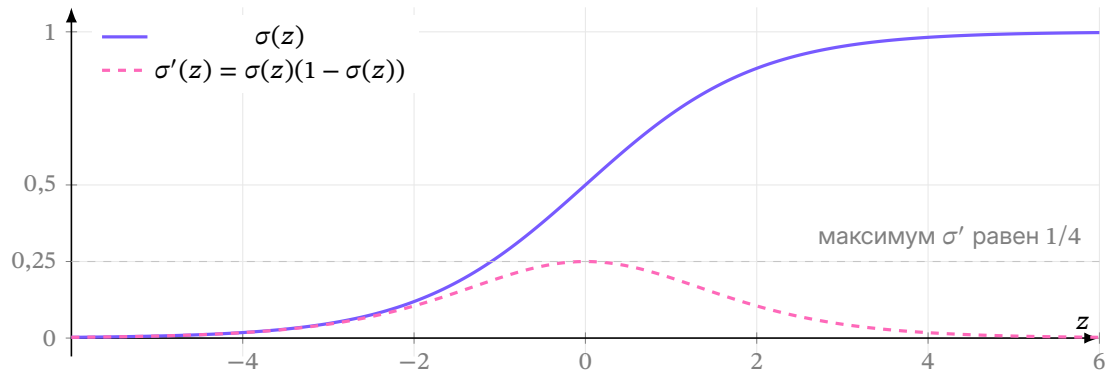


Рисунок 2. Сигмоида и её производная: производная не превосходит $1/4$, поэтому произведение таких множителей по слоям убывает экспоненциально по глубине

2. Функции активации

Правильный выбор функции активации — один из способов уменьшить затухание градиентов.

Функция	Затухание	Тождество	Гладкость	Сложность
$\sigma(z) = \frac{1}{1 + e^{-z}}$	FAIL	FAIL	OK	FAIL
$\tanh(z) = 2\sigma(2z) - 1$	FAIL	OK	OK	FAIL
$\text{ReLU}(z) = \max\{0, z\}$	OK	FAIL	FAIL	OK
$\text{LeakyReLU}(z) = \begin{cases} z, & z \geq 0, \\ \alpha z, & z < 0 \end{cases}$	OK	OK	FAIL	OK
$\text{ELU}(z) = \begin{cases} z, & z \geq 0, \\ e^z - 1, & z < 0 \end{cases}$	OK	OK	OK	FAIL
$\text{GELU}(z) = z\Phi(z)$	OK	OK	OK	FAIL
$\text{Swish}(z) = z\sigma(z)$	OK	OK	OK	FAIL
$\text{Mish}(z) = z \tanh(\text{softplus}(z))$	OK	OK	OK	FAIL
$\text{ReLU6}(z) = \min(\max(0, z), 6)$	OK	FAIL	FAIL	OK

Таблица 1. Свойства распространённых функций активации

2.1. ВОЗМОЖНОСТЬ СОХРАНИТЬ ТОЖДЕСТВО

Слой с сигмоидой на выходе не может реализовать тождественное преобразование на всей $\mathbb{R}$, поскольку его выход лежит в интервале $(0, 1)$. Для промежуточного слоя возможность не изменять уже найденные признаки является полезным свойством.

2.2. ВЫЧИСЛИТЕЛЬНАЯ СЛОЖНОСТЬ

Трансцендентные функции, например экспонента и логарифм, вычисляются с помощью численных аппроксимаций, редукции диапазона и специализированных инструкций. Обычно это дороже простых сравнений и умножений. Поэтому ReLU может вычисляться быстрее, чем сигмоида.

2.3. ГЛАДКОСТЬ

Функция гладкая, если её производная непрерывна. Гладкость позволяет получать более сильные теоретические оценки для некоторых методов оптимизации, но сама по себе не гарантирует фиксированного ускорения сходимости.

ReLU и LeakyReLU не дифференцируемы в нуле. На практике в этой точке выбирают одно из допустимых значений субградиента, поэтому градиентные методы всё равно применимы.

3. Нормализация

3.1. ЗАЧЕМ НОРМИРОВАТЬ ПРОМЕЖУТОЧНЫЕ ПРИЗНАКИ

При больших по модулю входах сигмоида попадает в насыщение, где её производная близка к нулю. Для ReLU также важен масштаб и знак входов:

- если все входы отрицательны, сигнал обрывается;
- если все входы положительны, исчезает нелинейность.

Нормализация удерживает промежуточные значения в более удобном диапазоне. Кроме того, числа умеренного масштаба численно устойчивее, а близкие масштабы признаков улучшают обусловленность функции потерь и уменьшают зигзаги градиентных методов.

3.2. BATCH NORMALIZATION

Пусть активации batch записаны по столбцам:

$$A^{(\ell)} \in \mathbb{R}^{n_\ell \times B},$$

где $A_{ji}^{(\ell)}$ — значение признака j у объекта i .

Для каждого признака считаются статистики по объектам batch:

$$\mu_j = \frac{1}{B} \sum_{i=1}^B A_{ji}^{(\ell)}, \quad \sigma_j^2 = \frac{1}{B} \sum_{i=1}^B (A_{ji}^{(\ell)} - \mu_j)^2.$$

Затем значения нормируются:

$$\tilde{A}_{ji}^{(\ell)} = \frac{A_{ji}^{(\ell)} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \quad \varepsilon > 0.$$

Константа ε предотвращает деление на ноль. После нормировки применяется обучаемое аффинное преобразование:

$$\hat{A}_{ji}^{(\ell)} = \gamma_j \tilde{A}_{ji}^{(\ell)} + \beta_j.$$

Параметры γ_j и β_j позволяют восстановить удобный масштаб и сдвиг — вплоть до тождественного преобразования при фиксированных статистиках.

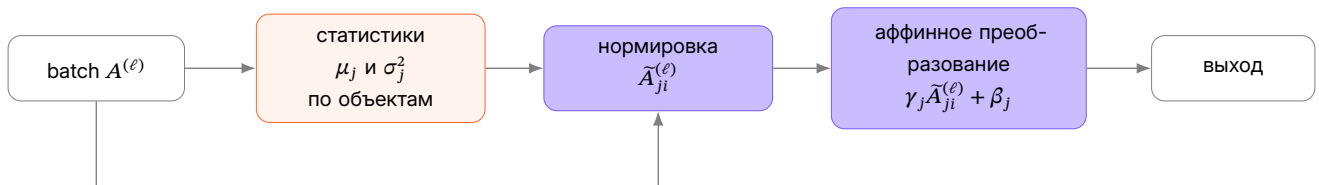


Рисунок 3. Batch Normalization: статистики считаются по объектам batch, поэтому нормировка каждого объекта зависит от всего batch, а обучаемые γ и β возвращают слою свободу масштаба и сдвига

Среднее и дисперсия зависят сразу от всех объектов mini-batch. Поэтому BatchNorm связывает их вычислительные графы: при обратном проходе градиент идёт не только через γ и β , но и через вычисление batch-статистик.

3.3. ОБУЧЕНИЕ И ТЕСТИРОВАНИЕ

Во время обучения BatchNorm использует статистики текущего batch. Параллельно обновляются скользящие оценки:

$$\hat{\mu}_j \leftarrow \alpha \hat{\mu}_j + (1 - \alpha) \mu_j, \quad \hat{\sigma}_j^2 \leftarrow \alpha \hat{\sigma}_j^2 + (1 - \alpha) \sigma_j^2, \quad 0 \leq \alpha < 1.$$

На тесте это позволяет обрабатывать объект независимо от состава текущего batch. При малом batch оценки μ_j и σ_j^2 могут быть шумными. Скользящие статистики не устраняют этот шум во время обучения: они нужны прежде всего для режима оценки.

Режим	Какие статистики используются
<code>model.train()</code>	Статистики текущего mini-batch; накопленные оценки обновляются
<code>model.eval()</code>	Накопленные оценки $\hat{\mu}_j$ и $\hat{\sigma}_j^2$

Таблица 2. Какие статистики использует BatchNorm в разных режимах

3.4. LAYER NORMALIZATION

LayerNorm считает статистики не по объектам batch, а по признакам одного объекта. Для каждого столбца i :

$$\mu_i^{\text{LN}} = \frac{1}{n_\ell} \sum_{j=1}^{n_\ell} A_{ji}^{(\ell)}, \quad (\sigma_i^{\text{LN}})^2 = \frac{1}{n_\ell} \sum_{j=1}^{n_\ell} (A_{ji}^{(\ell)} - \mu_i^{\text{LN}})^2.$$

Такой способ не требует достаточно большого batch для устойчивой оценки статистик.

3.5. ГДЕ РАЗМЕЩАТЬ НОРМАЛИЗАЦИЮ

BatchNorm добавляет обучаемые параметры и операции прямого и обратного проходов. Поэтому число и расположение слоёв нормализации остаётся инженерным выбором: их можно ставить после блоков, а не обязательно после каждого преобразования.

BatchNorm и Dropout решают разные задачи. Возможны сети только с BatchNorm, только с Dropout или с обоими видами слоёв; конкретную комбинацию выбирают экспериментально.

4. Инициализация весов

В оценке нормы градиента разные способы стабилизации отвечают за разные множители:

$$\|\nabla_{W^{(\ell)}} \mathcal{L}\| \lesssim \underbrace{\prod_{k=\ell+1}^L \|W^{(k)}\|}_{\text{инициализация}} \underbrace{\prod_{k=\ell}^{L-1} \|J_{g^{(k)}}(z^{(k)})\|}_{\text{активации и нормализация}} \underbrace{\|a^{(\ell-1)}\| \|\delta^{(L)}\|}_{\text{масштабы входа и выхода}}.$$

Если нормы весов слишком малы, градиент затухает; если слишком велики — взрывается. Поэтому начальный масштаб весов выбирают специально.

4.1. ИНИЦИАЛИЗАЦИЯ XAVIER/GLOROT

Рассмотрим линейную часть слоя без смещения:

$$z = Wx, \quad x \in \mathbb{R}^{n_{\text{in}}}, \quad z \in \mathbb{R}^{n_{\text{out}}}, \quad W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}.$$

Предположим, что компоненты входа и веса независимы, центрированы и имеют общие дисперсии:

$$\mathbb{E}[x_j] = 0, \quad \text{Var}(x_j) = \sigma_x^2, \quad \mathbb{E}[W_{ij}] = 0, \quad \text{Var}(W_{ij}) = \sigma_W^2.$$

Для прямого прохода

$$z_i = \sum_{j=1}^{n_{\text{in}}} W_{ij} x_j, \quad \text{поэтому} \quad \text{Var}(z_i) = n_{\text{in}} \sigma_W^2 \sigma_x^2.$$

Чтобы сохранить дисперсию активаций, требуется

$$\sigma_W^2 = \frac{1}{n_{\text{in}}}.$$

При обратном проходе

$$\nabla_x \mathcal{L} = W^\top \nabla_z \mathcal{L},$$

и аналогичный расчёт даёт условие сохранения дисперсии градиента:

$$\sigma_W^2 = \frac{1}{n_{\text{out}}}.$$

Если $n_{\text{in}} \neq n_{\text{out}}$, оба условия нельзя выполнить одновременно. Xavier/Glorot использует компромисс:

$$\sigma_W^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Нормальный вариант:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right),$$

где второй параметр — дисперсия. Равномерный вариант с той же дисперсией:

$$W_{ij} \sim \text{Unif}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right].$$



Область действия рассуждения

Это приближённый анализ первых итераций, основанный на независимости, центрированности и общей дисперсии. Инициализация даёт градиенту возможность пройти в глубину сети в начале обучения; дальнейшее поведение зависит от всей процедуры оптимизации.

4.2. ПОЧЕМУ ТОТ ЖЕ РАСЧЁТ ВОЗНИКАЕТ ДЛЯ ТАНН

Рассмотрим MLP с нечётной функцией g , для которой $g'(0) = 1$. Этим условиям удовлетворяет $\tanh$.

Предполагается следующее.

1. $b^{(\ell)} = 0$ для всех слоёв.
2. В начале обучения сеть находится в линейном режиме: $z_j^{(\ell)} \approx 0$, поэтому $g(z^{(\ell)}) \approx z^{(\ell)}$.
3. Веса слоя независимы от поступающих активаций и градиентов, имеют нулевое среднее, а координаты переменных на одном слое имеют одинаковые первые два момента.
4. Вход центрирован и имеет единичную дисперсию.

На прямом проходе

$$\text{Var}(z_j^{(\ell)}) = n_{\ell-1} \text{Var}(w_{jk}^{(\ell)}) \text{Var}(a_k^{(\ell-1)}).$$

Для сохранения масштаба активаций требуется

$$\text{Var}(w_{jk}^{(\ell)}) = \frac{1}{n_{\ell-1}}.$$

В линейном режиме $g'(z^{(\ell)}) \approx 1$, поэтому на обратном проходе

$$\text{Var}(\delta_k^{(\ell-1)}) = n_{\ell} \text{Var}(w_{jk}^{(\ell)}) \text{Var}(\delta_j^{(\ell)}).$$

Для сохранения масштаба градиентов требуется

$$\text{Var}(w_{jk}^{(\ell)}) = \frac{1}{n_{\ell}}.$$

Оба условия совпадают при одинаковой ширине соседних слоёв. В общем случае снова получается компромисс Xavier:

$$\text{Var}(W_{jk}^{(\ell)}) = \frac{2}{n_{\ell-1} + n_{\ell}}.$$

4.3. ОРТОГОНАЛЬНАЯ ИНИЦИАЛИЗАЦИЯ

Другой способ сохранить масштаб — инициализировать матрицу W так, чтобы её столбцы были ортонормированными:

$$W^T W = I.$$

Такую матрицу можно получить, например, сгенерировав случайную матрицу и ортогонализовав её методом Грама — Шмидта.

Для прямого прохода $z = Wx$

$$\|z\|_2^2 = x^T W^T W x = \|x\|_2^2.$$

Если W квадратная и ортогональная, то $W W^T = I$, поэтому норма градиента при умножении на W^T также сохраняется точно. Для прямоугольной матрицы ортонормированность столбцов гарантирует сохранение нормы на прямом проходе, но не нормы произвольного градиента на обратном проходе.

Ортогональность относится только к начальному состоянию. После обычного шага оптимизатора матрица, вообще говоря, перестаёт быть ортогональной.

4.4. ОГРАНИЧЕНИЯ ЧЕРЕЗ РЕПАРАМЕТРИЗАЦИЮ

Если условие должно выполняться на каждой итерации, возникает условная задача, например

$$\min_W \mathcal{L}(W) \quad \text{при условии} \quad W^T W = I.$$

Ограничение можно встроить в параметризацию:

$$W = h(V),$$

где V принимает произвольные значения, а $h(V)$ всегда удовлетворяет условию. Тогда решается безусловная задача

$$\min_V \mathcal{L}(h(V)),$$

и градиент проходит через дифференцируемое преобразование h .

4.4.1. Ортогональная матрица

Для квадратной матрицы можно взять

$$S(V) = V - V^T, \quad W(V) = \exp(S(V)).$$

Тогда $W(V)$ ортогональна и имеет определитель $+1$. Такая параметризация покрывает компоненту ортогональных матриц с определителем $+1$.

4.4.2. Ограниченный выход

Если выход должен лежать между -2 и 2 , сеть может сначала вычислить произвольное v , а затем

$$u = 2 \tanh v, \quad u \in (-2, 2).$$

4.4.3. Положительный параметр

Если сеть предсказывает дисперсию, можно выдать свободный параметр s и преобразовать его:

$$\sigma^2 = \exp(s) > 0 \quad \text{или} \quad \sigma^2 = \text{softplus}(s) = \log(1 + \exp s) > 0.$$

Так содержательное ограничение становится обычным дифференцируемым слоем вычислительного графа.

5. Остаточные и плотные связи

5.1. ДВА СПОСОБА ПЕРЕИСПОЛЬЗОВАТЬ ПРИЗНАКИ

Прокидывание связей позволяет повторно использовать признаки предыдущих слоёв.

1. **Остаточная связь** — поэлементно сложить вход с результатом блока.
2. **Плотная связь** — конкатенировать вход с результатом блока.

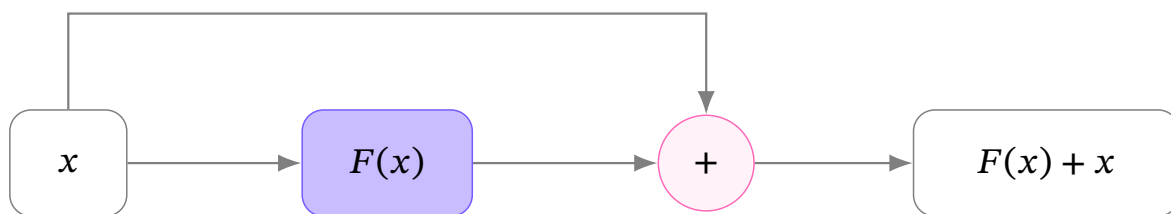
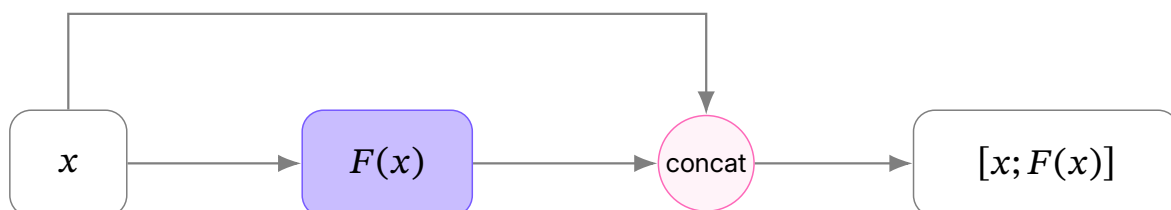
Остаточная связь**Плотная связь**

Рисунок 4. Два способа прокинуть вход мимо блока: остаточная связь складывает его с выходом блока, плотная — приписывает выход блока к входу

5.2. ОСТАТОЧНАЯ СВЯЗЬ И ГРАДИЕНТ

Пусть блок вычисляет $F(x)$. Остаточная связь задаёт

$$y = F(x) + x.$$

Для сложения размерности $F(x)$ и x должны совпадать. Дифференциал выхода:

$$dy = dx + J_F(x) dx = (I + J_F(x))dx.$$

Поэтому

$$\boxed{\nabla_x \mathcal{L} = \nabla_y \mathcal{L} + J_F(x)^\top \nabla_y \mathcal{L} .}$$

Тожественная ветвь добавляет прямой путь для $\nabla_y \mathcal{L}$ без умножения на якобиан нелинейной ветви. Это не даёт безусловной гарантии от затухания во всех ситуациях, но стабилизирует обучение глубоких вычислительных графов.

Похожий аддитивный путь встретится дальше в рекуррентных сетях, в частности в LSTM, хотя исторически в этой сфере используется другая терминология.

5.3. ЧТО ДАЮТ ПРОКИНУТЫЕ СВЯЗИ

Прокинутые связи дают несколько эффектов:

- сокращается путь обратного распространения и ослабляется затухание градиентов;
- признаки предыдущих слоёв не теряются и могут использоваться повторно;
- может устраняться симметрия полносвязных слоёв;
- ландшафт функции потерь может становиться более гладким.

Дополнительные материалы: [гипотеза о симметрии](#) и [работа о ландшафте функции потерь](#).

6. Dropout

Dropout случайно выключает часть нейронов во время обучения. Для активаций слоя разыгрывается независимая маска

$$r_j \sim \text{Bernoulli}(1 - p),$$

где p — вероятность обнулить нейрон. Выход Dropout:

$$a_{\text{drop}}^{(\ell)} = \frac{r \odot a^{(\ell)}}{1 - p}.$$

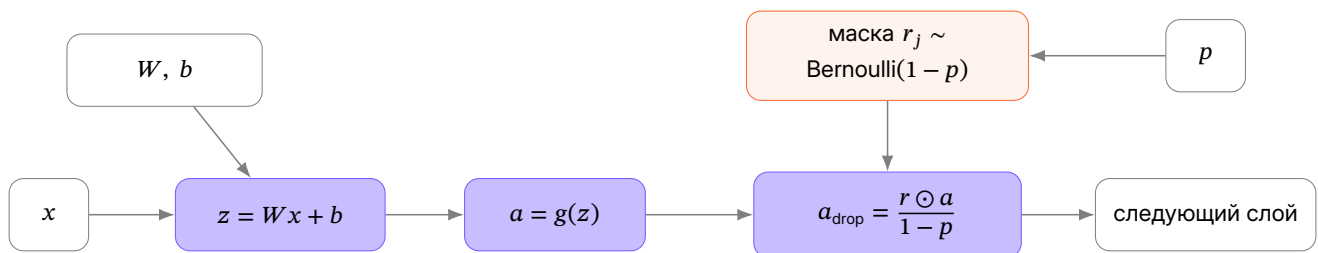


Рисунок 5. Dropout как вершина вычислительного графа: маска r разыгрывается независимо от данных, а деление на $1 - p$ возвращает выходу прежний масштаб

Деление на $1 - p$ называется **inverted dropout** и сохраняет условное математическое ожидание:

$$\mathbb{E}_r \left[\frac{r \odot a^{(\ell)}}{1 - p} \mid a^{(\ell)} \right] = a^{(\ell)}.$$

Без этого множителя ожидаемый масштаб во время обучения был бы в $1 - p$ раз меньше, чем при использовании всех нейронов на тесте.

6.1. КОАДАПТАЦИЯ НЕЙРОНОВ

Если один нейрон уже выучил сильный признак, нейроны следующего слоя могут начать почти полностью полагаться на него. При случайном выключении доступный набор признаков меняется от batch к batch. Сеть вынуждена использовать разные признаки и не может постоянно полагаться на одну и ту же совместную конфигурацию нейронов.

Градиент проходит через оставшиеся активными связи. Отключённые нейроны на этой итерации получают нулевой градиент. Маска r и вероятность p не обучаются.

6.2. ОБУЧЕНИЕ И ТЕСТИРОВАНИЕ

Режим	Поведение Dropout
<code>model.train()</code>	Маска разыгрывается заново, часть активаций обнуляется
<code>model.eval()</code>	Все нейроны включены; дополнительное масштабирование не требуется

Таблица 3. Поведение Dropout в разных режимах модели

Переключение режима модели важно даже тогда, когда градиенты вычислять не требуется.

Родственная идея — **DropConnect**; отдельно она не разбирается.

7. Стохастическая и неявная регуляризация

7.1. ШУМ СТОХАСТИЧЕСКОГО ГРАДИЕНТА

Стохастические градиентные методы используются не только для экономии вычислений. Шумная оценка градиента может давать регуляризующий эффект.

Рассмотрим линейную регрессию по одному признаку:

$$\mathcal{L}(w) = \frac{1}{N} \sum_{i=1}^N (wx_i - y_i)^2,$$

где $x_i, y_i, w \in \mathbb{R}$. При размере batch, равном одному, на шаге выбирается объект i и выполняется обновление:

$$w \leftarrow w - 2\eta x_i (wx_i - y_i).$$

Каждое слагаемое функции потерь — парабола по w . Если существует хорошее общее решение,

вершины этих парабол лежат около общей области.

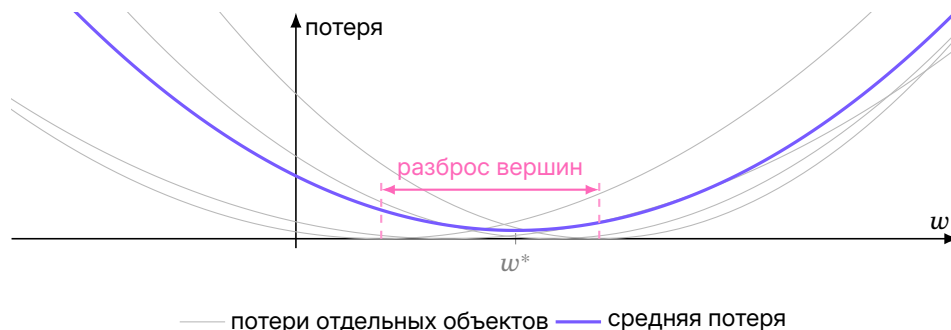


Рисунок 6. Потери отдельных объектов — параболы со слегка разными вершинами. Далеко от оптимума все шаги ведут примерно в одну сторону, а вблизи него направления отдельных объектов расходятся

- Далеко от этой области стохастические шаги в среднем направлены к оптимуму.
- Вблизи оптимума градиенты отдельных объектов могут указывать в разные стороны.
- При постоянном ненулевом темпе обучения SGD обычно осциллирует в окрестности оптимума.
- При подходящем убывающем темпе обучения сходимость возможна.

С точки зрения точной оптимизации шум мешает. Но в машинном обучении важна не только ошибка на обучающей выборке, поэтому шум стохастических градиентов может работать как неявная регуляризация.

В невыпуклом ландшафте глубоких сетей этот шум иногда помогает покидать седловые области и узкие минимумы. Эффект зависит от задачи, размера batch и темпа обучения.

7.2. ШИРОКИЕ МИНИМУМЫ

Интуиция здесь такая: минимум, устойчивый к небольшим изменениям параметров и стохастическому шуму, оказывается широким. Такой минимум может соответствовать области, которая подходит многим объектам и лучше обобщается. При этом широкий минимум сам может быть зашумлён и содержать много локальных минимумов.



L_2 -штраф и weight decay

L_2 -регуляризация и различие между ней и weight decay относятся к следующей лекции об оптимизации. Здесь достаточно помнить, что это отдельный способ контролировать масштаб весов; подробный вывод остаётся в конспекте по оптимизации.

8. Главное

Проблема	Приёмы из лекции
Градиент затухает или взрывается	Выбор активации, нормализация, инициализация, остаточные связи
Активации уходят в насыщение или имеют неудобный масштаб	BatchNorm, LayerNorm
Начальный масштаб сигнала нестабилен	Xavier/Glorot, ортогональная инициализация
Ограничение должно выполняться всегда	Репараметризация
Сеть слишком зависит от конкретного набора признаков	Dropout
Нужен прямой путь для признаков и градиента	Residual- и dense-связи
Требуется неявный регуляризующий эффект	Шум стохастических градиентов

Таблица 4. Какая проблема каким приёмом закрывается



Ключевая идея

Устойчивость глубокой сети определяется тем, как через её слои проходят активации и градиенты. Регуляризация отвечает за то, чтобы найденное решение не опиралось на слишком хрупкую конфигурацию признаков или параметров. Многие методы из лекции одновременно влияют и на устойчивость оптимизации, и на обобщающую способность модели.